

Análisis de Clases de Water Emblem: Principios de POO y Desafíos de Desarrollo

Este documento describe la funcionalidad principal de las clases proporcionadas para el juego "Water Emblem", destacando cómo se aplican los principios de la Programación Orientada a Objetos (POO) y los desafíos encontrados durante su desarrollo.

1. Funcionamiento General de las Clases

A continuación, se presenta un resumen de la responsabilidad de cada clase:

- **GamePanel (Paquete Main):**
 - Es el **corazón del juego**. Hereda de JPanel de Swing y actúa como el bucle principal del juego.
 - Gestiona el estado general del juego (menú, jugando, victoria).
 - Inicializa y coordina a todos los demás "manejadores" y entidades.
 - Se encarga de la lógica de actualización (update()) y el dibujo (paintComponent()) de todos los elementos del juego.
 - Controla la música y la transición entre niveles.
- **ManejadorEntidades (Paquete Main):**
 - Su función principal es **gestionar todas las entidades** que existen en el mundo del juego (unidades aliadas y enemigos).
 - Es responsable de la creación inicial de estas entidades para cada nivel.
 - Coordina la actualización y el dibujo de todas las entidades en cada ciclo del juego.
 - Permite añadir y eliminar entidades del juego.
- **ManejadorTeclas (Paquete Main):**
 - Es el encargado de **capturar y procesar la entrada del teclado** del jugador.
 - Mantiene el estado de las teclas presionadas (si están abajo o acaban de ser soltadas).
 - Proporciona métodos para que otras clases consulten el estado de teclas específicas.
- **MusicManager (Paquete Main):**
 - Gestiona la **reproducción de la música** de fondo del juego.
 - Permite iniciar la música de un nivel específico y detenerla.
- **TurnManager (Paquete Main):**
 - Implementa la **lógica del sistema de turnos** del juego.
 - Determina si es el turno del jugador o del enemigo.
 - Reinicia las acciones de las unidades al inicio de cada turno.

- Coordina el paso de turno entre jugadores y enemigos.
- Mantiene un registro de qué unidades han actuado en el turno actual.
- **Entidad (Paquete entidad):**
 - Es una **clase abstracta** que sirve como base para todos los objetos que tienen una posición en el mundo del juego y pueden ser actualizados y dibujados.
 - Define propiedades comunes como la posición en el mundo (mundoX, mundoY), velocidad, y referencias al GamePanel y ManejadorTeclas.
 - Declara métodos abstractos update() y draw() que deben ser implementados por sus subclases.
- **Unidad (Paquete entidad):**
 - Representa a las **unidades controlables por el jugador**.
 - Extiende la clase Entidad, añadiendo propiedades específicas como nombre, estadísticas (Stats), inventario (Inventario), arma equipada (Armas), clase (Clases), y estado de selección.
 - Contiene la lógica de movimiento específica de la unidad, incluyendo el cálculo de su rango de movimiento.
 - Gestiona la animación de la unidad y su estado de acción (si ha movido o actuado en el turno).
- **Enemigo (Paquete entidad):**
 - Representa a las **unidades controladas por la IA (enemigos)**.
 - Extiende la clase Entidad. Aunque su implementación específica de update() no fue proporcionada, se espera que contenga la lógica de comportamiento de la IA (movimiento, ataque).
- **Cursor (Paquete entidad):**
 - Representa el **cursor visual del jugador** en el mapa.
 - Extiende la clase Entidad para gestionar su posición en el mundo.
 - Es responsable de actualizar qué Unidad está seleccionada actualmente basándose en su posición.
 - Maneja la lógica de selección de unidades y la interacción con el menú de acciones.

2. Principios de la Programación Orientada a Objetos (POO)

El diseño de estas clases muestra una aplicación clara de varios principios de POO:

Herencia

La herencia es fundamental en este diseño para reutilizar código y establecer una jerarquía de tipos:

- **Unidad hereda de Entidad:** Esto permite que Unidad comparta las propiedades básicas de posición, velocidad y acceso al GamePanel y ManejadorTeclas que toda entidad posee.
- **Enemigo hereda de Entidad:** Similar a Unidad, los Enemigos obtienen las características fundamentales de una entidad.
- **Cursor hereda de Entidad:** El cursor también es un objeto en el mundo con una posición y necesita ser actualizado y dibujado, por lo que hereda de Entidad.

Polimorfismo

El polimorfismo se observa principalmente a través de la clase abstracta Entidad:

- Los métodos update() y draw() de la clase Entidad son abstractos. Esto significa que cada subclase (Unidad, Enemigo, Cursor) **debe implementar su propia versión** de cómo se actualiza y cómo se dibuja.
- El ManejadorEntidades mantiene una lista de Entidades (List<Entidad>). Cuando llama a e.update() o e.draw(g2) para cada elemento e en la lista, el sistema Java determina automáticamente qué versión específica de update() o draw() (la de Unidad, Enemigo o Cursor) debe ejecutar en tiempo de ejecución.

Interfaces

- **GamePanel implementa Runnable:** Esto le permite ser ejecutado en un hilo (Thread), lo que es esencial para el bucle de juego que se ejecuta de forma concurrente.
- **ManejadorTeclas implementa KeyListener:** Esto le permite escuchar y responder a los eventos del teclado de Java.

Cohesión

- **ManejadorTeclas:** Su única responsabilidad es manejar la entrada del teclado.
- **MusicManager:** Su única responsabilidad es la reproducción de música.
- **TurnManager:** Se centra exclusivamente en la lógica de turnos.
- **ManejadorEntidades:** Se dedica a la gestión de las entidades.
- **Entidad:** Define las características base para *todas* las entidades.
- **Unidad / Enemigo / Cursor:** Cada una se enfoca en las responsabilidades específicas de su tipo de entidad.

Acoplamiento

- ManejadorTeclas y MusicManager tienen un acoplamiento relativamente bajo con el resto del sistema. Simplemente proporcionan servicios o estados que

otras clases consumen.

- **GamePanel:** Como orquestador central, tiene un acoplamiento moderado con casi todas las demás clases de "manejador" y con el Cursor.
- **Entidad y sus subclases (Unidad, Enemigo, Cursor):** Están acopladas a GamePanel y ManejadorTeclas (ya que se les pasan en sus constructores y las usan internamente).
- **ManejadorEntidades y TurnManager:** Están acoplados a GamePanel y a la jerarquía de Entidad para poder realizar sus funciones de gestión.

3. Desafíos y Soluciones en el Desarrollo

Durante el desarrollo de "Water Emblem", nos enfrentamos a varios problemas que requirieron depuración y rediseño para asegurar el correcto funcionamiento del juego.

Problema: Enemigos Incapaces de Derrotar Unidades Aliadas

Inicialmente, notamos que las unidades enemigas no eran capaces de "matar" completamente a las unidades aliadas del jugador; solo podían dejarlas con muy poca vida. Esto era un comportamiento inesperado y frustrante.

- **Hipótesis Iniciales:** Al principio, sospechamos que el problema radicaba en una implementación defectuosa de la inteligencia artificial de los enemigos o en un mal funcionamiento del ManejadorEntidades al procesar el daño o la eliminación de unidades.
- **Causa Raíz:** Después de una investigación exhaustiva, descubrimos que el verdadero culpable era el **método setter de los puntos de vida (HP) en la clase Stats**. Este método realizaba varias verificaciones internas que, bajo ciertas condiciones de cómo se aplicaba el daño, impedían que el HP de una unidad descendiera por debajo de un umbral mínimo, evitando así su derrota.
- **Solución:** Se ajustó la lógica del método setHP() en la clase Stats para permitir que los puntos de vida pudieran reducirse a cero o menos, asegurando que las unidades pudieran ser derrotadas correctamente según la lógica de combate.

Problema: TurnManager y Movimiento de Enemigos

En las primeras etapas de implementación de las unidades enemigas, surgieron dos problemas críticos: el TurnManager dejaba de funcionar correctamente y los enemigos no eran capaces de moverse.

- **Síntomas:** El juego se estancaba, los turnos no avanzaban como se esperaba y las unidades enemigas permanecían inmóviles en el mapa.
- **Causa Raíz y Enfoque de Solución:** Se determinó que la implementación inicial

de la clase Enemigo carecía de la planificación necesaria en cuanto a su lógica de update() y su interacción con el TurnManager. De manera similar, el TurnManager mismo tenía deficiencias en cómo manejaba la transición del turno del jugador al turno del enemigo y cómo notificaba las acciones de las unidades.

Como solución se optó por una estrategia de rediseño y refactorización.

- **Clase Enemigo:** Se rehizo la clase Enemigo con una planificación más detallada de su comportamiento, asegurando que su método update() contuviera la lógica adecuada para el movimiento y las acciones de la IA, y que interactuara correctamente con el TurnManager para notificar cuándo había completado su acción.
- **Clase TurnManager:** Se revisó y refactorizó la clase TurnManager para asegurar que las transiciones de turno fueran robustas y que los métodos para el paso al turno enemigo (startEnemyTurn(), endEnemyTurn(), notificarAccionEnemigo()) funcionaran de manera predecible y correcta, reiniciando los estados de acción de las unidades apropiadamente.